

The lutorch_ex library

architecture, cartridges, and the 48k champion-topology sweep

(draft)

2026-10-04 (DRAFT — refine next session)

Abstract

`lutorch_ex` is a clean, cartridge-based re-implementation of the lookup-table FFN with zero imports from the old `spiky.lutorch`. This report documents (1) its architecture, (2) each cartridge’s math and usage, and (3) a seven-configuration champion-topology sweep at 48k steps. **[TBD — abstract — finalise once all 7 runs land.]**

1 General architecture

The library factors the LUT-FFN into four layers: an immutable *geometry*, a cartridge *contract*, concrete *cartridges* (the lookup math), and a dense *wrapper* that presents a plain $d_{\text{model}} \rightarrow d_{\text{model}}$ map.

Geometry — LUTSpec. An immutable dataclass fixing heads/tables/anchor-pairs/widths: `h_in`, `h_out`, `tph`, `nap`, `d_in`, `d_out`, with derived `n_groups` = $\max(h_{\text{in}}, h_{\text{out}})$, `n_cells` = 2^{nap} (cells per table, K), `in_features` = $h_{\text{in}} d_{\text{in}}$, `out_features` = $h_{\text{out}} d_{\text{out}}$, `n_tables` = `n_groups` · `tph`.

Base — ManifestoLUT. The abstract multi-head LUT. It owns the learnable cell tables $W \in \mathbb{R}^{G \times \text{tph} \times K \times d_{\text{out}}}$ ($G = \text{n_groups}$) and the addressing machinery: anchors, the per-group cell index c from the sign pattern of anchor-pair differences (`_addresses`, which uses only spec/anchors/powers, never the weight *values*), the routing invariant `_route` ($[B, G, d_{\text{out}}] \rightarrow [B, h_{\text{out}}, d_{\text{out}}]$: identity when $h_{\text{out}} = G$, group-sum when $h_{\text{out}} = 1$), and cell total-variation `cell_tv_penalty`. `forward` compiles `_forward_impl` with `torch.compile` on CUDA (eval always; train when the cartridge sets `_COMPILE_TRAIN`); CPU stays eager.

Default init (faithful, OLD-matching). Cell tables initialise per head from a CPU generator seeded (`seed + h + 1`), $\text{Uniform}[-\sigma, +\sigma]$ with $\sigma = \text{weight_init_std}$ (realised std $\sigma/\sqrt{3} \approx 5.77\text{e-}4$ at the 10^{-3} default), reproducing the OLD LightMHL draw bit-for-bit. `ProjectionMHL` initialises `compress` $\sim \mathcal{N}(0, 0.02)$ and `zeroes decompress` (the FFN starts as a zero contribution).

1.1 ProjectionMHL: the ProjectionLUT wrapper

`ProjectionMHL` is the only module the surrounding transformer sees: it turns an opaque cartridge into a plain $d_{\text{model}} \rightarrow d_{\text{model}}$ FFN, so the (cheap, discrete) lookup runs in a narrow per-head space while the block sees a dense map. It treats the cartridge as a black box reached only through the shape contract.

Protocol. $z = \text{compress}(x)$ reshaped to $[B, h_{\text{in}}, d_{\text{in}}]$; $y = \text{cartridge}(z)$ of shape $[B, h_{\text{out}}, d_{\text{out}}]$; $\text{out} = \text{decompress}(y)$. Here **compress** is a Linear $d_{\text{model}} \rightarrow \text{in_features}$ and **decompress** a Linear $\text{out_features} \rightarrow d_{\text{model}}$; either side may be switched off (Identity), with the matching width required to already equal d_{model} .

Init and first-backward warmup. $\text{compress} \sim \mathcal{N}(0, 0.02)$; **decompress** is *zeroed*. Because $\partial \text{out} / \partial y = W_{\text{dec}} = 0$ at init, the very first backward sends *no* gradient through the main loss to the cartridge or to **compress**; on step 0 only **decompress** itself (and, separately, the cartridge tables via the TV term) receive gradient, and the input-side path “warms up” once **decompress** moves off zero. This reproduces OLD’s zero-contribution FFN at init. Both inits are meta-guarded so the deployment skeleton build allocates nothing.

Two capabilities it mediates.

- **SupportsDecompressBake:** a cartridge may expose **decompress_scale()** returning a length-**out_features** vector; **ProjectionMHL** folds it into the **decompress** matrix on every forward (scaling column k of $W_{\text{dec}} = \text{scaling cartridge output channel } k$, with no extra op). This is how a quantised cartridge’s per-channel scale is moved into the dense projection.
- **SupportsDeploymentExport:** **to_deployment()** for compaction. The general, cartridge-agnostic deploy path (§1) pairs it with a **from_deployment** rebuilder (via **deploy_registry**). **ProjectionMHL** is the seam at which the power-of-two output scale is folded, once, into **decompress** at export time.

Native-op resolution. Fused cartridges reach a tier-2 CUDA fast path through a single choke point, `_native_ops.native_manager()`, which now resolves: (1) the co-located JIT extension `lutorch_ex_lprojection` (vendored `lprojection` / anchor-pairs kernels, built via `torch.utils.cpp_extension.load` from `cartridges/csrc/`); then (2) the pure/tier-1 torch path. The int8 power-of-two read is a second co-located JIT extension, `lutorch_ex_pow2_int8_read`. All paths fall back to pure torch when CUDA/nvcc/ninja are unavailable. **[TBD — confirm final fallback wording vs the merged PR #140 (lutorch_cuda fallback removed).]**

Deployment — export_deployment / load_deployment. An inference-only, cartridge-agnostic path (`deploy.py`). `export_deployment(model, "m.lxq")` walks the model, and for each **ProjectionMHL** whose cartridge implements **SupportsDeploymentExport.to_deployment()** it writes a compact payload; the per-(group,channel) power-of-two output scale is folded *exactly* into a copy of the **decompress** weight. `load_deployment(path, make_skeleton, device)` builds the architecture on the **meta** device (no fp32 allocation), then swaps each compacted FFN for the inference cartridge named by a stored format tag (via the leaf **deploy_registry**: tag $\rightarrow$ **from_deployment**), so the fp32 LUT table is never allocated. The `.lxq` file is a safetensors container; footprint drops $\sim 4\times$ for the quantised LUT (int8 **packed** table replaces the fp32 master + exponent buffer).

2 Cartridges

Shared read. Each group g addresses one cell $c_g \in [0, K)$ per table from the sign pattern of its anchor-pair differences; the read sums the selected cell rows over the **tph** tables, optionally score-weighted, then `_routes` $[B, G, d_{\text{out}}] \rightarrow [B, h_{\text{out}}, d_{\text{out}}]$. **[TBD — cross-check all per-cartridge math against doc/lutorch/lut_mechanisms.tex (§LightMHL, §confidence forms).]**

2.1 ConfidenceLUT

Gen-3 scored read. A learned-margin confidence score weights each table’s cell contribution. With margins u (anchor-pair differences) and $m = |u|$, the per-table score is

$$s = \left(\sum_j m_j \right) \exp\left(\gamma \sum_j \log \sigma(\beta m_j) \right),$$

$\beta = e^{\log \beta}$, $\gamma = e^{\log \gamma}$ learnable (per layer; g frozen at 0 in the champion). `read_top_n=2` blends the two most-confident cells with a τ -temperature softmax over the deciding margin; `read_top_n=1` reads the single scored cell. Gradients flow to W , β , γ (and τ for $n=2$). **[TBD — write the exact $n=2$ blend weights ($v, 1-v$) and the τ form from `confidence.py` / `lut_mechanisms` §confidence forms.]**

```
from spiky.lutorch_ex import LUTSpec, ConfidenceLUT, ProjectionMHL
spec = LUTSpec(h_in=8, h_out=8, tph=64, nap=8, d_in=48, d_out=48)
cart = ConfidenceLUT(spec, seed=1, read_top_n=2, beta_init=2.0, gamma_init=1.0,
                    read_tau_init=0.5, read_tau_learnable=True)
ffn = ProjectionMHL(cart, d_model=512) # drop-in d_model -> d_model FFN
```

2.2 QuantisedConfidenceLUT (extended)

The `p2_int8` quant-aware twin of `ConfidenceLUT`. Same confidence score/blend, but the read is quantisation-aware with a straight-through estimator (STE):

- **Cell tables** are fake-quantised to int8 with a per-(group,channel) *power-of-two* scale: for group g , channel k , an integer exponent $e_{g,k}$ (`head_chan_exponents`) gives $\hat{W} = \text{round}(W/2^e) \cdot 2^e$, clamped to int8; STE passes the identity gradient to the fp32 master W .
- **Blend weights** are rounded to powers of two: $2^{k'}$ and $2^{k'-q}$ with $k' = \text{round}(\log_2 s - c_q)$ and $q = \text{round}(2|u_{j*}|/(\tau \ln 2))$ (stop-grad exponents; the STE carries the exact $(s \cdot v, s \cdot (1-v))/s$ gradient).
- **Integer read** (`forward_int` / the deployed path): int8 cell rows, int32 shift-add in fixed point (shift = 6, `FIXED_POINT_SHIFT`), the accumulator in units 2^{-6} , then scaled by 2^{e-6} per (group,channel). This equals the fake-quant forward value to round-off.

Deployment bake. `to_deployment()` emits the int8 packed table and the per-out-feature scale 2^{e-6} ; `export_deployment` folds that pure-power-of-two multiplier *exactly* into a copy of the `decompress` weight (scaling column k by 2^{e-6} only touches the fp32 exponent field). The deployed cartridge then returns the *raw* int32 accumulator and reports `decompress_scale()`=None (the scale is static in the stored weight). Requires `read_top_n=2` and $h_{\text{out}} = G$ (the champion); fan-in would need a pre-route scale. **[TBD — verify the exact c_q constant and the `quant_cfg` (bits/offset/Q/lo/hi) against `_pow2.py`; cite `lut_ablation` §quant.]**

```
from spiky.lutorch_ex import QuantisedConfidenceLUT, ProjectionMHL, export_deployment,
load_deployment
cart = QuantisedConfidenceLUT(spec, seed=1, quant_mode="p2_int8", read_top_n=2,
                             beta_init=2.0, gamma_init=1.0, read_tau_init=0.5)
ffn = ProjectionMHL(cart, d_model=512)
# ... train ...
export_deployment(model, "m.lxq") # int8 bake + fold pow2 scale into decompress
dep = load_deployment("m.lxq", make_skeleton, device="cuda").eval() # fp32 table never
    allocated
```

2.3 FusedManifestoHardLUT

Hard value read $\sum_t W[c_t]$ (argmax cell per table), fused. Tier-2 native path uses the lprojection non-smooth backward; tier-1 uses `embedding_bag`. **[TBD — exact forward/backward from `manifesto_hard.py` + `_native_ops.NativeHard`.]**

```
from spiky.lutorch_ex import FusedManifestoHardLUT, ProjectionMHL
ffn = ProjectionMHL(FusedManifestoHardLUT(spec, seed=1), d_model=512)
```

2.4 FusedManifestoSoftLUT

Soft temperature-blended read; native path uses `lprojection_forward_smooth` + its `nal-smooth` backward. **[TBD — soft-blend math from `manifesto_soft.py`.]**

2.5 FusedSoftSignHardLUT / FusedSoftSignSmoothLUT

Gen-2 soft-sign cartridges. The two-cell blend weight is $w = \sigma(-2\rho/T_{\text{sel}})$ with $\rho = |m|/(T_{\text{soft}} + |m|)$ (deciding margin m , learned temperatures $T_{\text{soft}}, T_{\text{sel}}$); hard uses the straight-through value, smooth the true blend. The surrogate’s m/T gradients are a one-launch `softsign_surrogate_grad` kernel (tier-1: eager). **[TBD — confirm against `softsign*.py` + `_native_softsign.py`; lut_mechanisms has the Gen-2 forms.]**

```
from spiky.lutorch_ex import FusedSoftSignHardLUT, FusedSoftSignSmoothLUT, ProjectionMHL
ffn = ProjectionMHL(FusedSoftSignSmoothLUT(spec, seed=1), d_model=512)
```

3 The 48k champion-topology sweep

Seven configurations, all at the champion topology and regularisation: $h=8$, $\text{tph}=64$, $\text{nap}=8$, $d_{\text{in}}=d_{\text{out}}=48$ (6 blocks, $\approx 68.2\text{M}$ params), cell-TV $\lambda=10$, head-dropout 0.2, 48,000 steps, cosine schedule (10% warmup, floor $0.1 \times \text{lr}$), seed 1, faithful small-std default init. wandb project `Spiky`, group `lutorch_ex`. **[TBD — confirm `lr/weight_decay/batch` from the `config.json` and add them here.]**

3.1 Regularisation and training recipe

Two regularisers are part of the champion recipe (applied on `lutorch_ex`, not the skeleton), both at whole-cell / whole-table granularity.

Cell total-variation ($\mathcal{R}_{\text{TV}}, \lambda=10$). View each table’s $K=2^{\text{nap}}$ cells as the vertices of an nap -dimensional hypercube, reshaping the table to $[G \cdot \text{tph}, \underbrace{2, \dots, 2}_{\text{nap}}, d_{\text{out}}]$. The penalty sums the squared difference (channels summed in) over every Hamming-1 cell pair — realised as a size-2 `diff` along each of the nap bit-axes, each pair counted once — normalised by the pair count:

$$\mathcal{R}_{\text{TV}} = \frac{1}{n_{\text{tables}} \text{nap} 2^{\text{nap}-1}} \sum_{(c, c') \text{ Hamming-1}} \|W_c - W_{c'}\|_2^2.$$

It is added to the loss as $\lambda \mathcal{R}_{\text{TV}}$ ($\lambda=10$) through a separate backward, is differentiable w.r.t. W only, and adds nothing to the forward graph. Two cells that are Hamming-1 neighbours are

addressed by inputs differing in a *single* anchor-sign bit, so penalising their squared difference smooths the table: a one-bit flip of the address changes the read little — a locality/smoothness prior over the discrete lookup. (Bit-for-bit the reference `LightMultiHeadLUT.cell_tv`.)

Head (table) dropout ($r = 0.2$). Inverted dropout at *whole-table* granularity: one independent Bernoulli per (sample, group=head, table) on the $[B, G, \text{tph}]$ axis, keep probability $1 - r$, survivors rescaled by $1/(1 - r)$ so the expected read is unchanged; dropped tables contribute 0. It is train-time and grad-only (eval reads all tables). By randomly removing whole tables within a head each step it discourages co-adaptation among a head’s `tph` tables. The same keep-mask multiplies the backward, so the weight-gradient and the input-grad carriers inherit the per-table factor.

Configuration	cartridge / mode	wandb run	final
ConfidenceLUT $n=2$	scored, float	lutorch_ex_abl47_conf_1004_1148	
QuantConfidence $n=2$	scored, p2_int8	lutorch_ex_abl47_quant_1004_1148	
ConfidenceLUT $n=1$	scored, float, $n=1$	lutorch_ex_abl47_conf_n1_vm_1004_1806	[TBD — queued]
FusedManifestoHard	hard	lutorch_ex_abl47_fmh_wk_1004_1510	[TBD — running]
FusedManifestoSoft	soft	lutorch_ex_abl47_fms_wk_1004_1510	[TBD — queued]
FusedSoftSignHard	soft-sign, hard	lutorch_ex_abl47_fss_hard_wk_1004_1510	[TBD — queued]
FusedSoftSignSmooth	soft-sign, smooth	lutorch_ex_abl47_fss_smooth_vm_1004_1806	[TBD — running]

Status (2026-10-04). 2 done (ConfidenceLUT $n=2$, QuantConfidence $n=2$), 2 running (FusedSoftSignSmooth on the VM, FusedManifestoHard on the worker), 3 queued (ConfidenceLUT $n=1$; FusedManifestoSoft, FusedSoftSignHard). [TBD — insert the `val_bpb` learning-curve figure (7 curves) once the runs complete; pull per-step from wandb or the local `metrics.csv`.]